Java Application CI/CD Pipeline

Jenkins · Maven · SonarQube · Docker · Helm · Argo CD · Kubernetes

Name	Bhagyashree
Course / Batch	
Submission date	
GitHub repository	https://github.com/Bhagyashree2993/java-cicd
Docker Hub image	https://hub.docker.com/r/guvibhagyashreekhandare/java-app

1. Project Overview

This project builds a production-oriented CI/CD pipeline for a Java (Spring Boot) application. Jenkins handles Continuous Integration: it builds and tests the code, checks quality with SonarQube, builds and scans a Docker image, and pushes it to Docker Hub. Argo CD handles Continuous Delivery using GitOps: Jenkins writes the new image tag into the Helm chart in Git, and Argo CD automatically deploys that version to Kubernetes.

Application

- **Spring Boot 3 REST API (Java 17)** with endpoints /, /api/greet and /api/add
- **Health probes** at /actuator/health/liveness and /readiness, and Prometheus metrics
- **9 automated tests** (JUnit 5 + MockMvc) with JaCoCo code coverage

Architecture

Java Application → Docker → Kubernetes

CI/CD Workflow

```
GitHub → Jenkins → Maven → Test → SonarQube → Docker → Container Registry → Helm → Git → Argo CD → Kubernetes
```

Tools Used

Tool	Role in the project
GitHub	Stores the source code, Jenkinsfile, Helm chart and Argo CD manifest
Jenkins	CI server; runs the pipeline defined in the Jenkinsfile
Maven	Compiles the code, runs unit tests and packages the JAR
SonarQube	Static code analysis and coverage; Quality Gate stops bad code
Docker	Packages the app as a non-root container image
Trivy	Scans the image for HIGH/CRITICAL vulnerabilities
Docker Hub	Container registry that stores versioned images
Helm	Packages the Kubernetes manifests (Deployment, Service, HPA, PDB)
Argo CD	GitOps tool; syncs the cluster with the Helm chart in Git
Kubernetes	Runs the application (Docker Desktop cluster)

Pipeline Stages

Jenkins stage	What it does
Checkout	Pulls the code from GitHub; skips commits marked [skip ci] to avoid loops
Build & Unit Test	mvn clean verify: compile, run tests, create JAR and coverage report
SonarQube Analysis	Sends code and coverage to SonarQube
Quality Gate	Waits for SonarQube result (via webhook); fails the build if the gate fails
Docker Build	Builds image tagged <build number>-<git commit>
Image Scan (Trivy)	Reports HIGH/CRITICAL vulnerabilities in the image

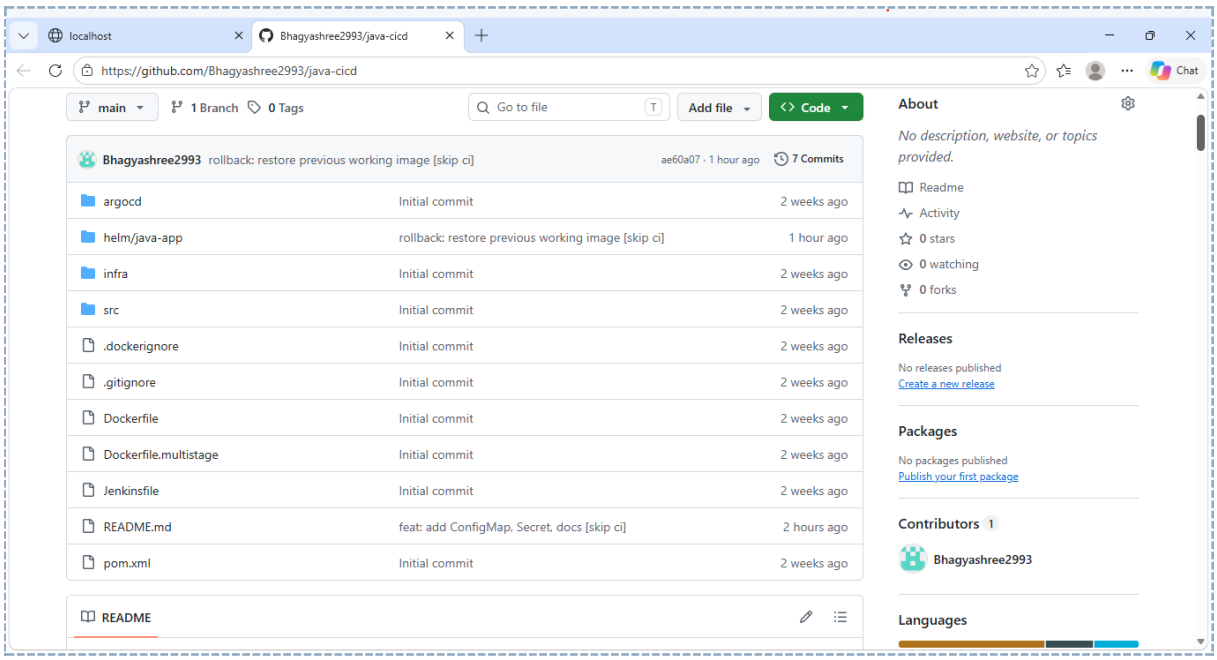
Jenkins stage	What it does
Push to Registry	Logs in and pushes the image to Docker Hub
Update Helm values (GitOps)	Writes the new tag into helm/java-app/values.yaml and pushes a [skip ci] commit
Argo CD (outside Jenkins)	Detects the Git change and rolls the new version out to Kubernetes

2. Source Code and Environment Setup

Screenshot 1: GitHub repository with project files

The repository contains the Spring Boot source code, pom.xml, Dockerfile, Jenkinsfile, the Helm chart (helm/java-app) and the Argo CD Application (argocd/application.yaml).

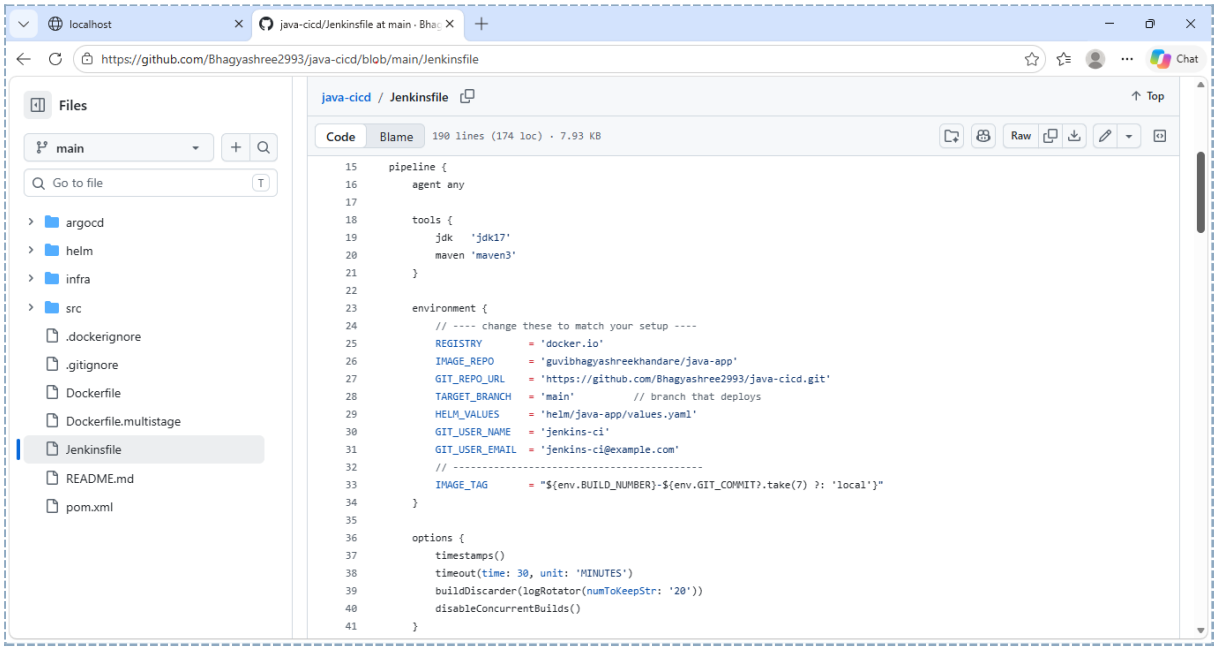
How to capture: *Open https://github.com/Bhagyashree2993/java-cicd and capture the file list.*

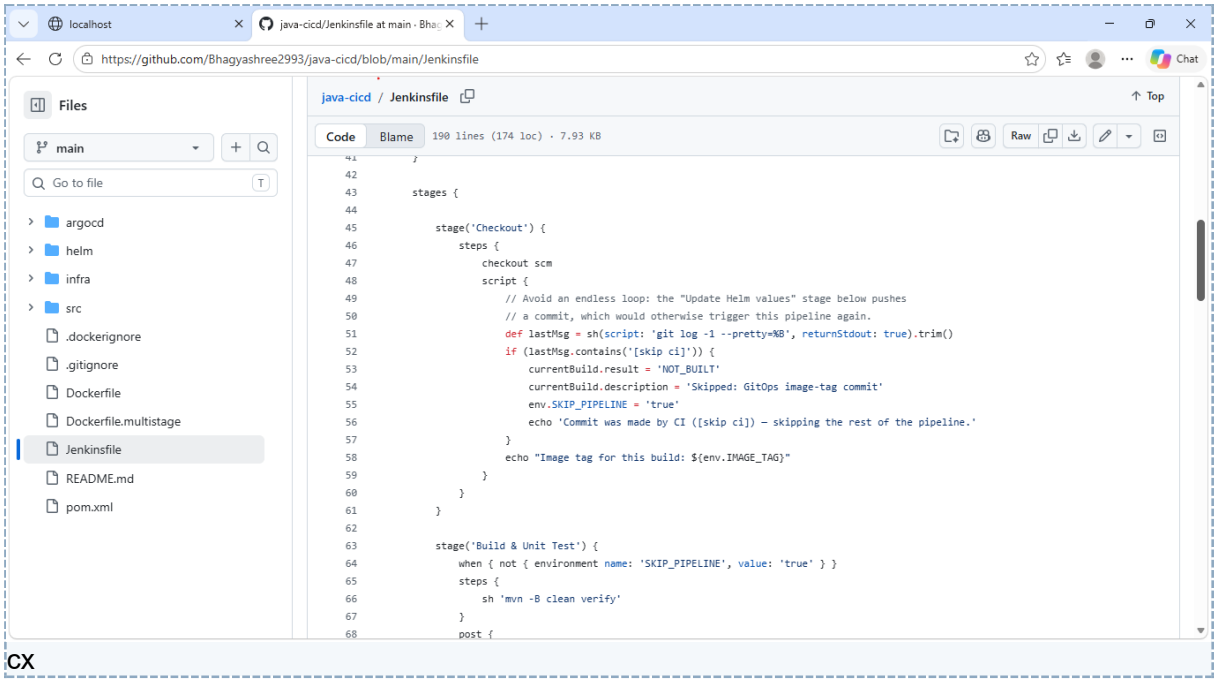


Screenshot 2: Jenkinsfile in the repository

The Jenkinsfile defines all pipeline stages, tools and credentials used by Jenkins.

How to capture: *Open Jenkinsfile on GitHub and capture the top (environment and first stages).*



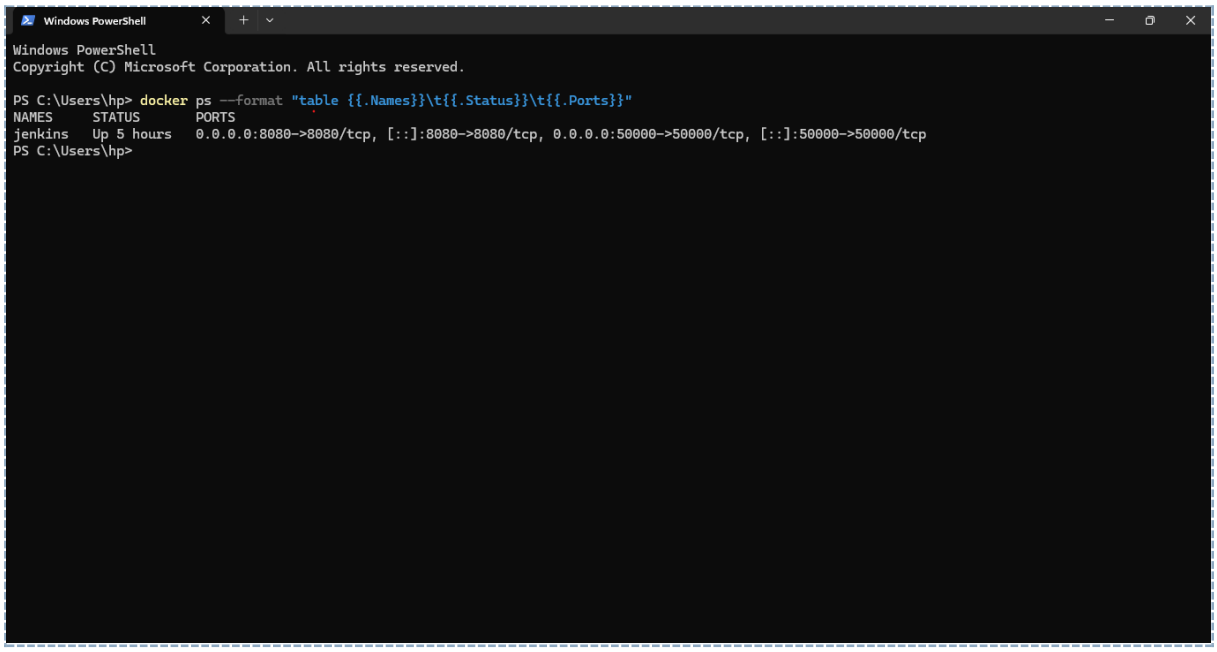


CX

Screenshot 3: Jenkins, SonarQube and PostgreSQL running in Docker

Jenkins and SonarQube run locally as Docker containers started with docker compose (infra/docker-compose.yml).

How to capture: *PowerShell:* `docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"`

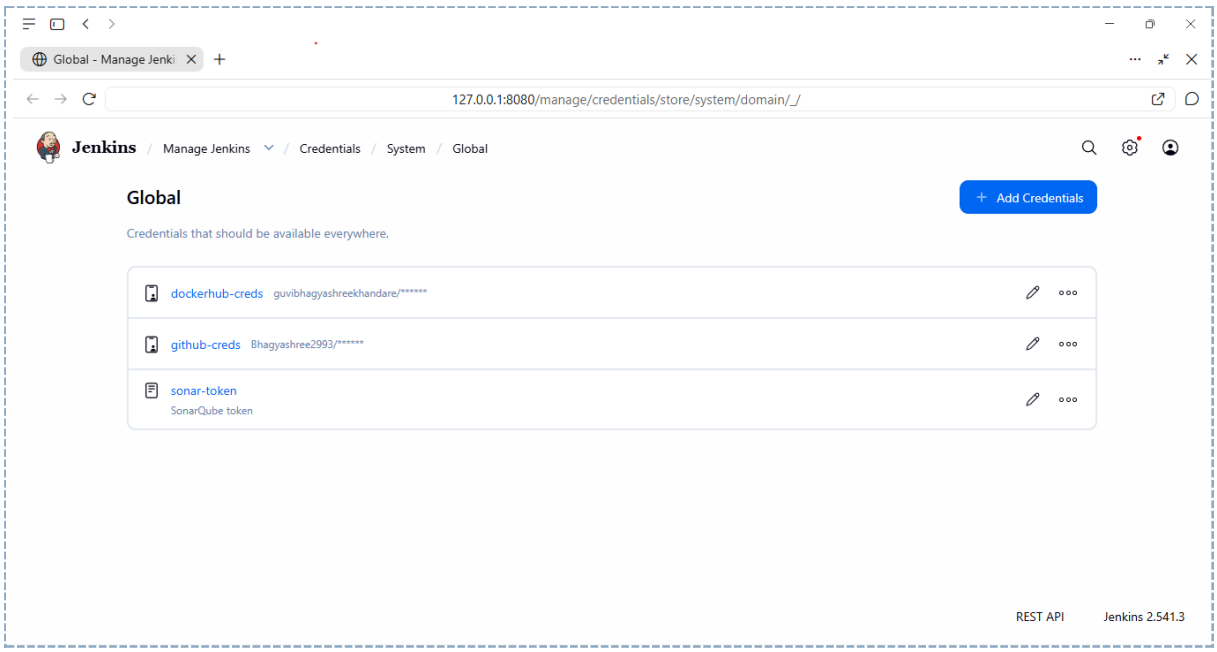


3. Jenkins Configuration

Screenshot 4: Jenkins credentials

Three credentials are stored securely in Jenkins: sonar-token (SonarQube), dockerhub-creds (Docker Hub) and github-creds (GitHub). Secrets are never written in the Jenkinsfile.

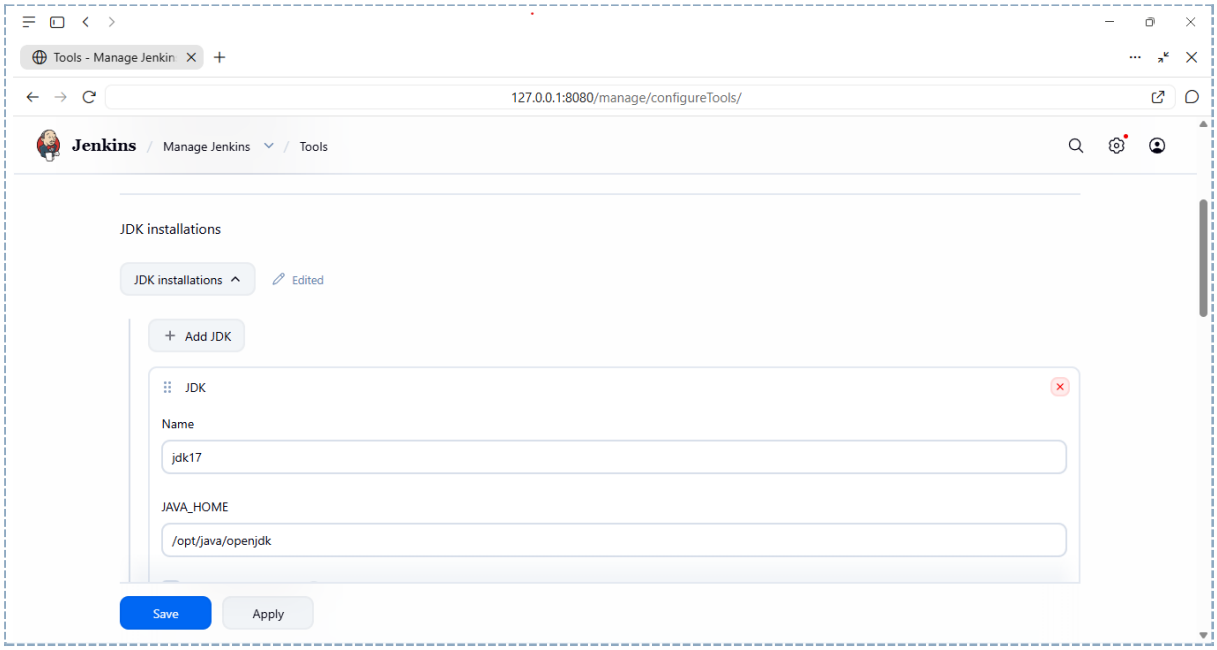
How to capture: *Manage Jenkins → Credentials → System → Global credentials.*

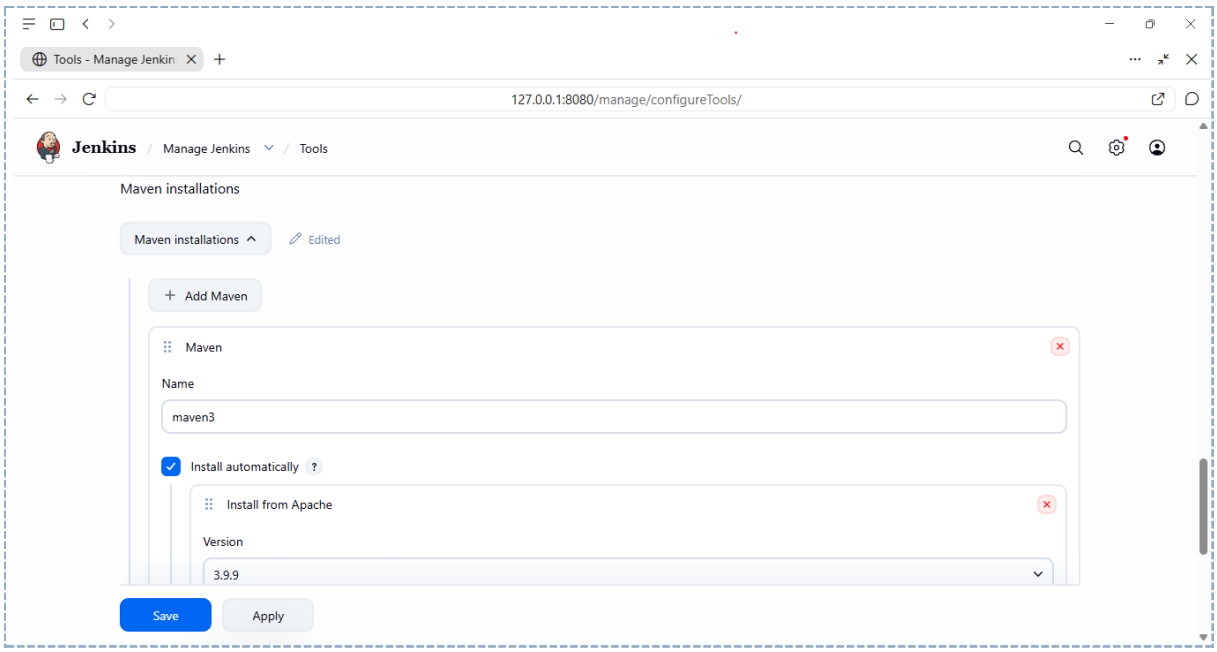


Screenshot 5: Jenkins tools: JDK 17 and Maven

The pipeline uses the tools named jdk17 and maven3, configured once in Jenkins.

How to capture: *Manage Jenkins → Tools (scroll to JDK and Maven installations).*

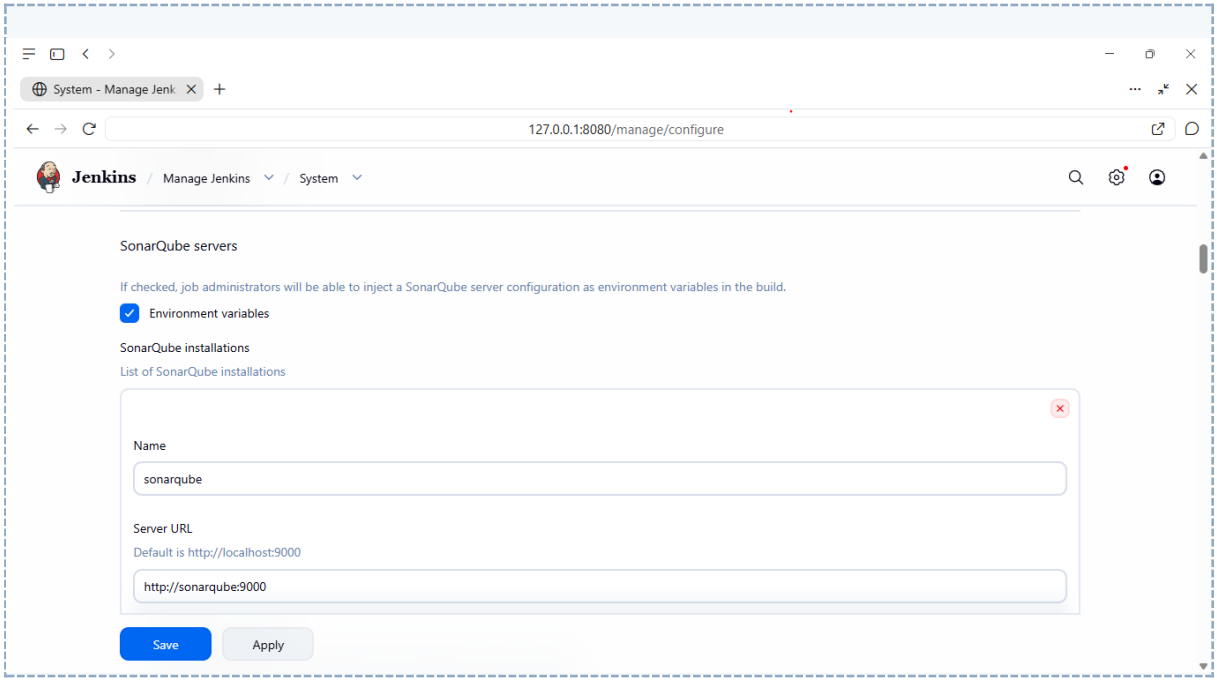




Screenshot 6: SonarQube server connected to Jenkins

Jenkins knows the SonarQube server as "sonarqube" at http://sonarqube:9000 and authenticates with the sonar-token credential.

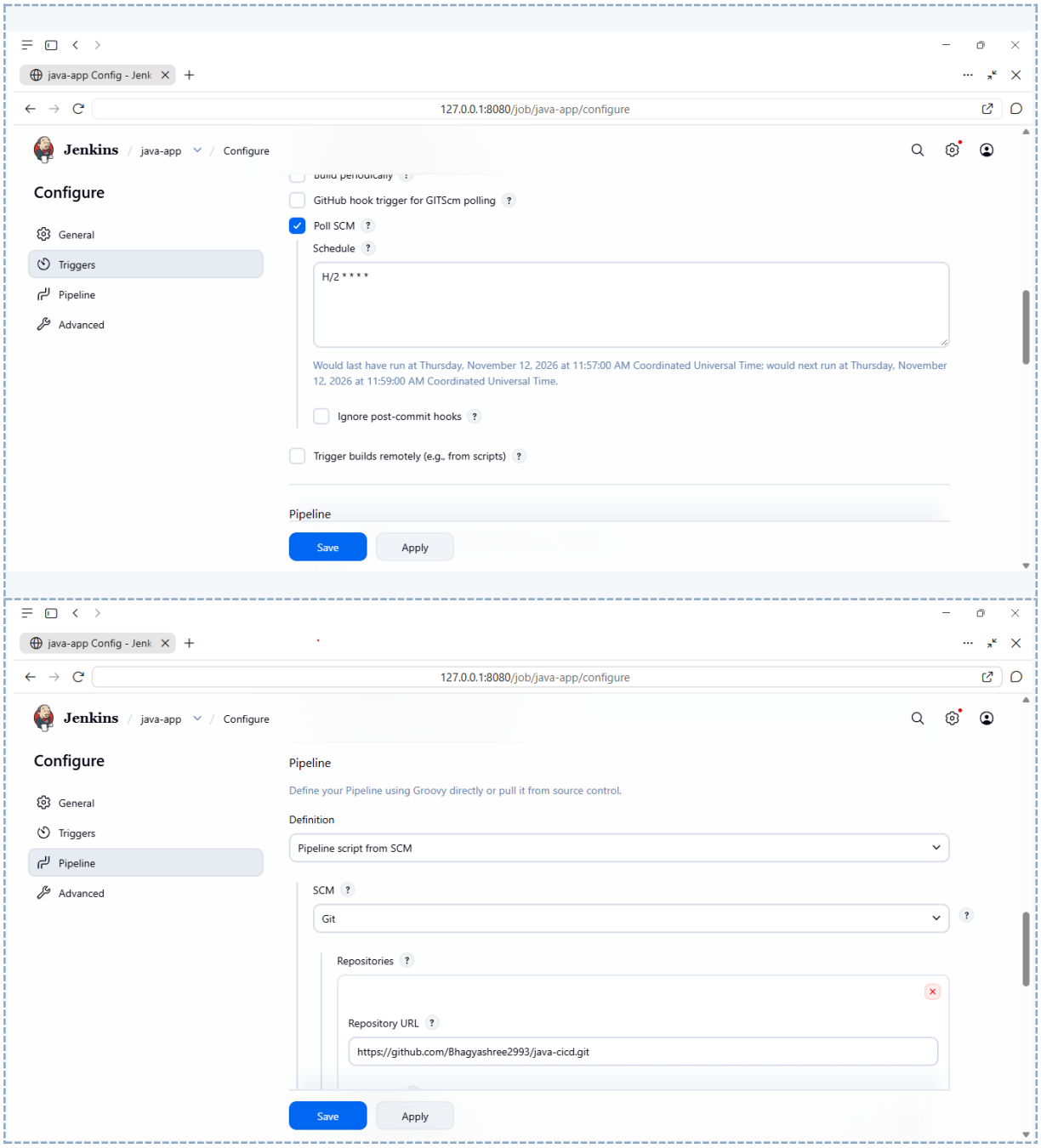
How to capture: *Manage Jenkins → System → SonarQube servers.*

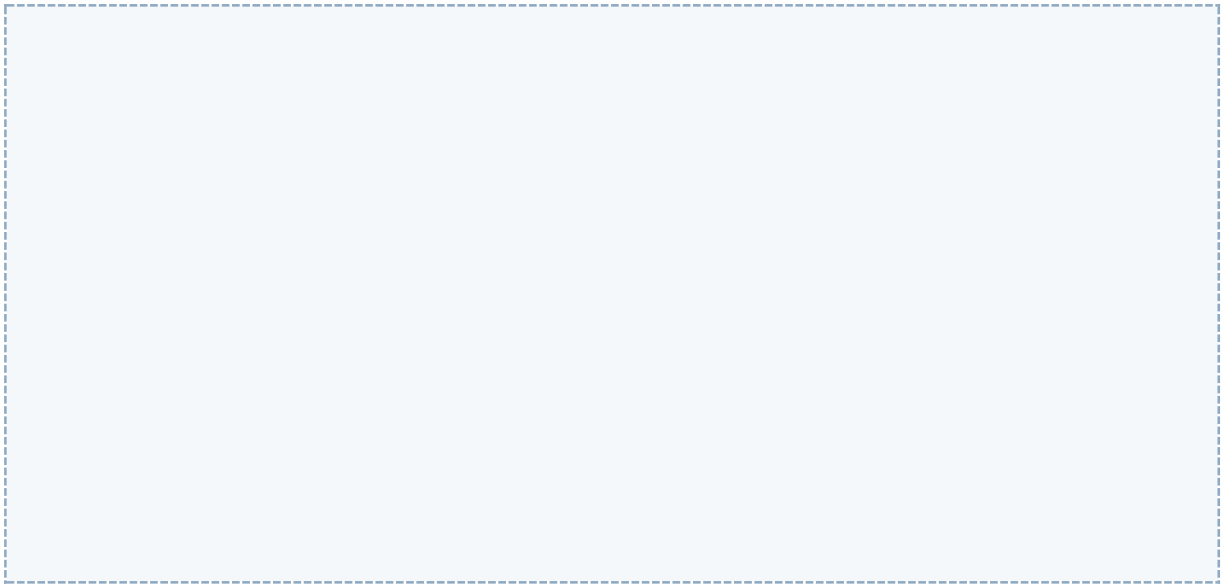


Screenshot 7: Pipeline job configuration

The java-app job reads the Jenkinsfile from GitHub (branch main) and polls GitHub every 2 minutes for new commits.

How to capture: *java-app → Configure → Triggers and Pipeline sections.*



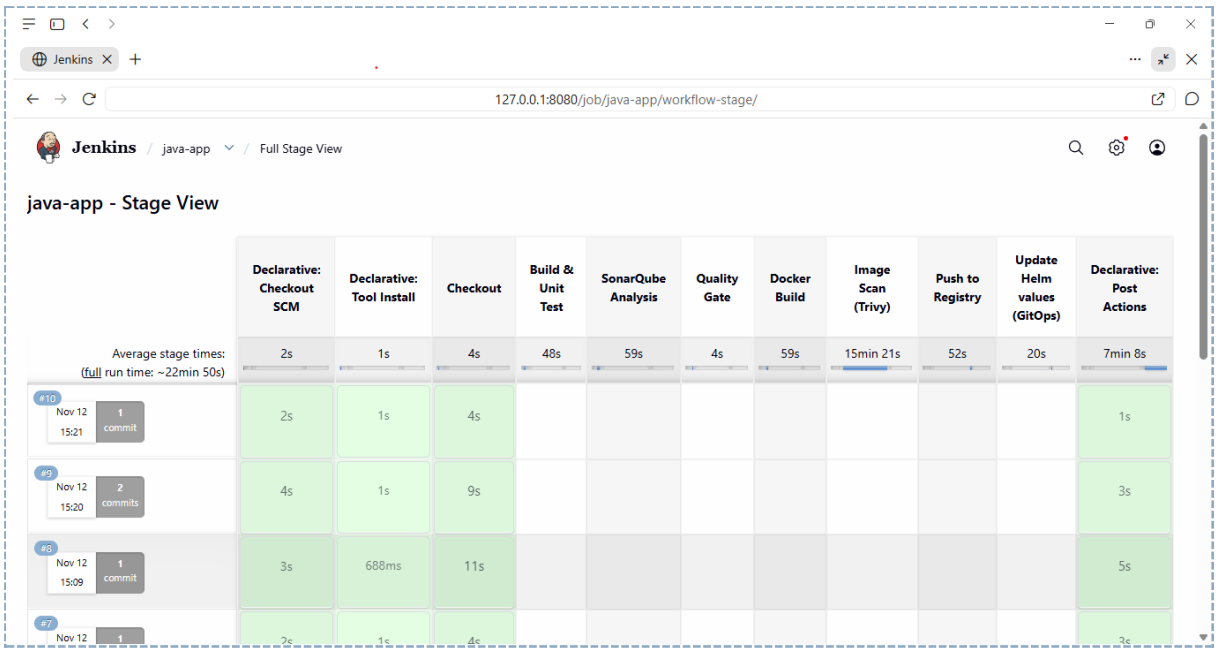


4. Continuous Integration Run

Screenshot 8: Successful pipeline: stage view

All stages completed successfully: checkout, build and test, SonarQube analysis, Quality Gate, Docker build, Trivy scan, push to Docker Hub, and the GitOps update.

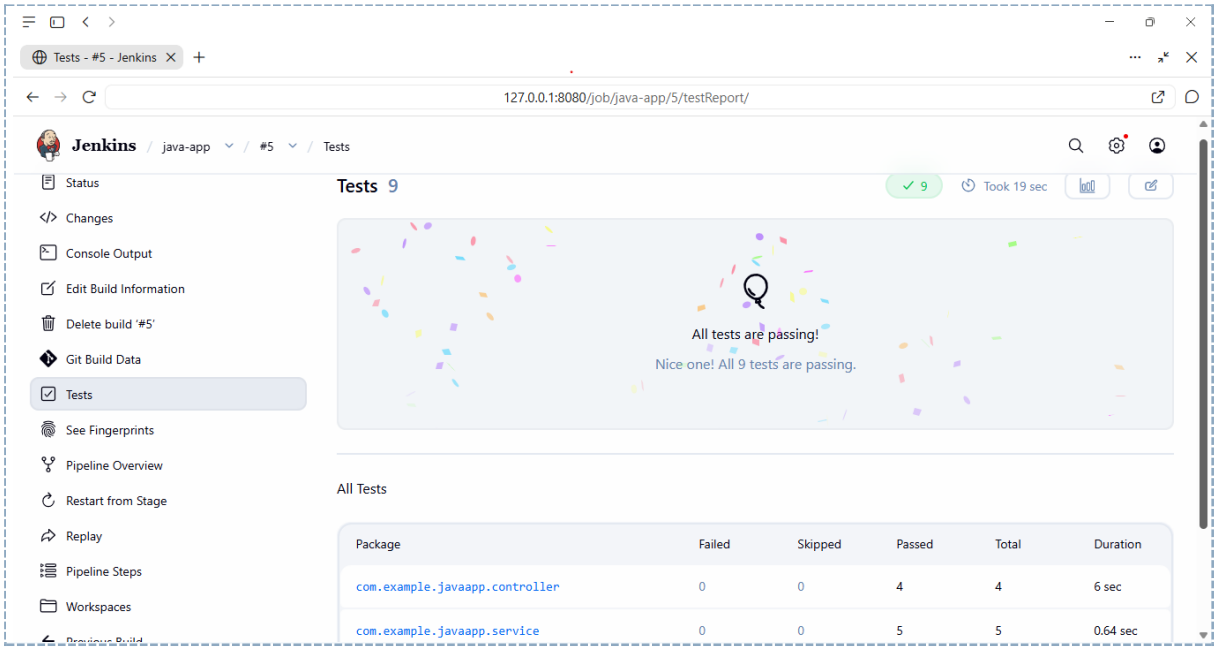
How to capture: *Jenkins → java-app → the successful build (all stages green).*

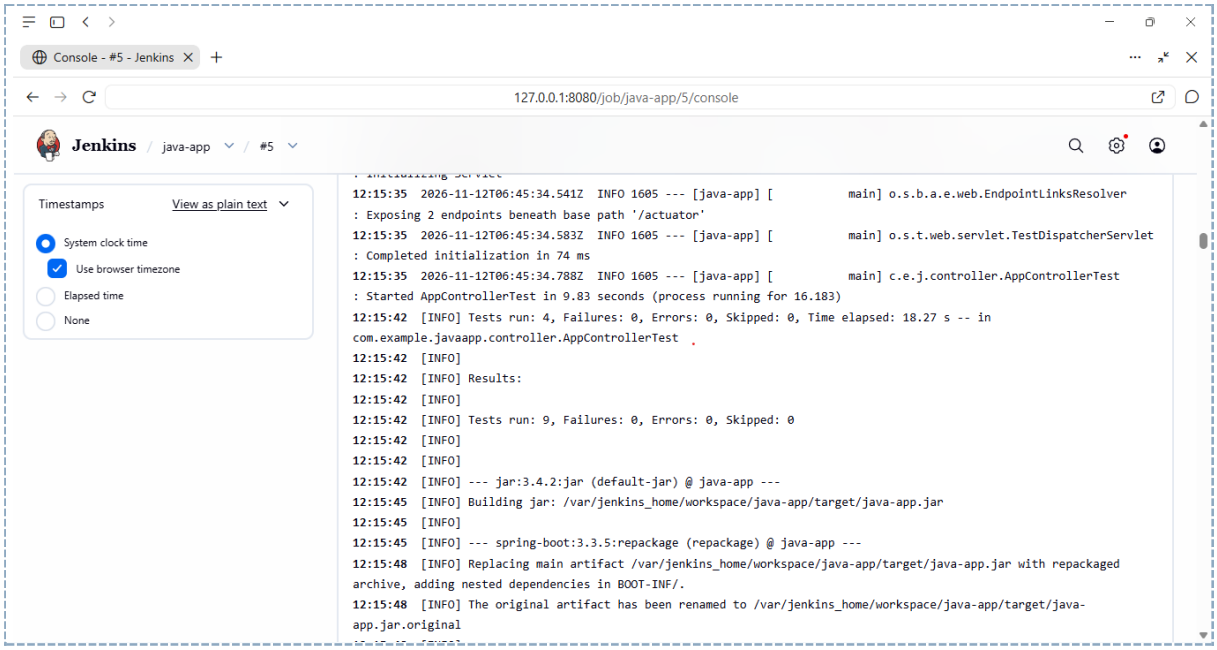


Screenshot 9: Unit tests passed

Maven ran 9 tests with 0 failures and 0 errors. Jenkins records the results with the JUnit plugin.

How to capture: *Build → Console Output, find "Tests run: 9, Failures: 0" (or open Test Result).*

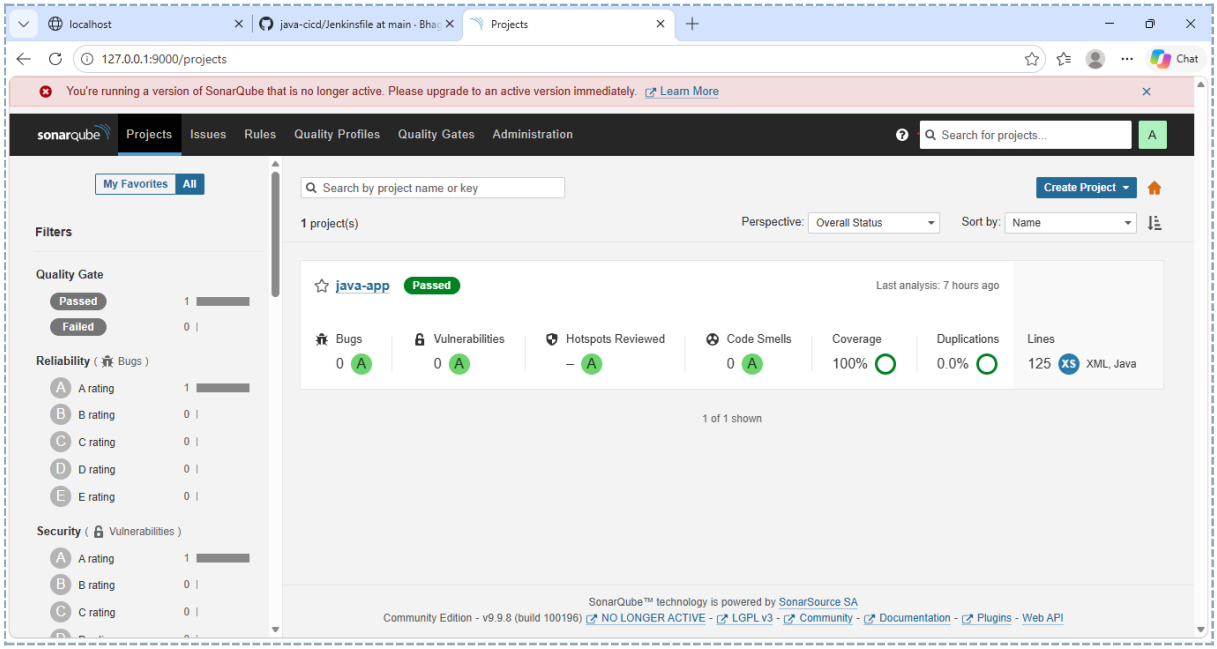




Screenshot 10: SonarQube Quality Gate passed

SonarQube analysed the code and the Quality Gate returned "Passed", so the pipeline continued.

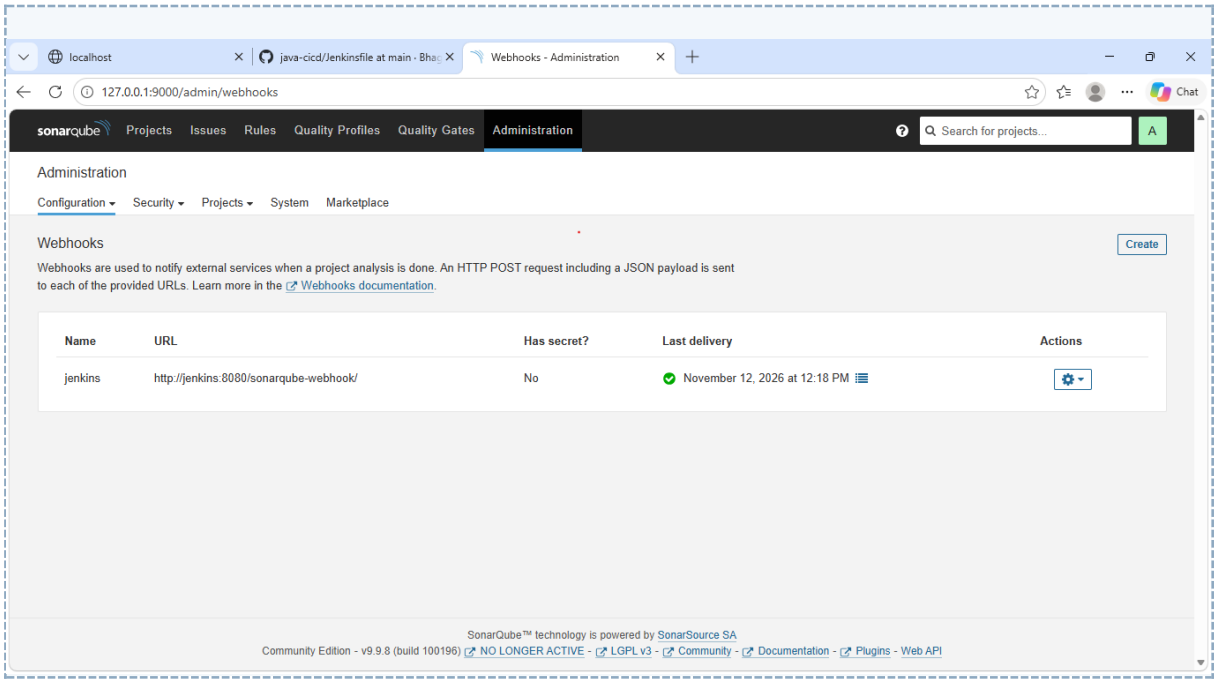
How to capture: *SonarQube (http://127.0.0.1:9000) → Projects → java-app.*



Screenshot 11: SonarQube webhook to Jenkins

The webhook notifies Jenkins when analysis finishes, so the Quality Gate stage does not have to wait for a timeout.

How to capture: *SonarQube → Administration → Configuration → Webhooks.*

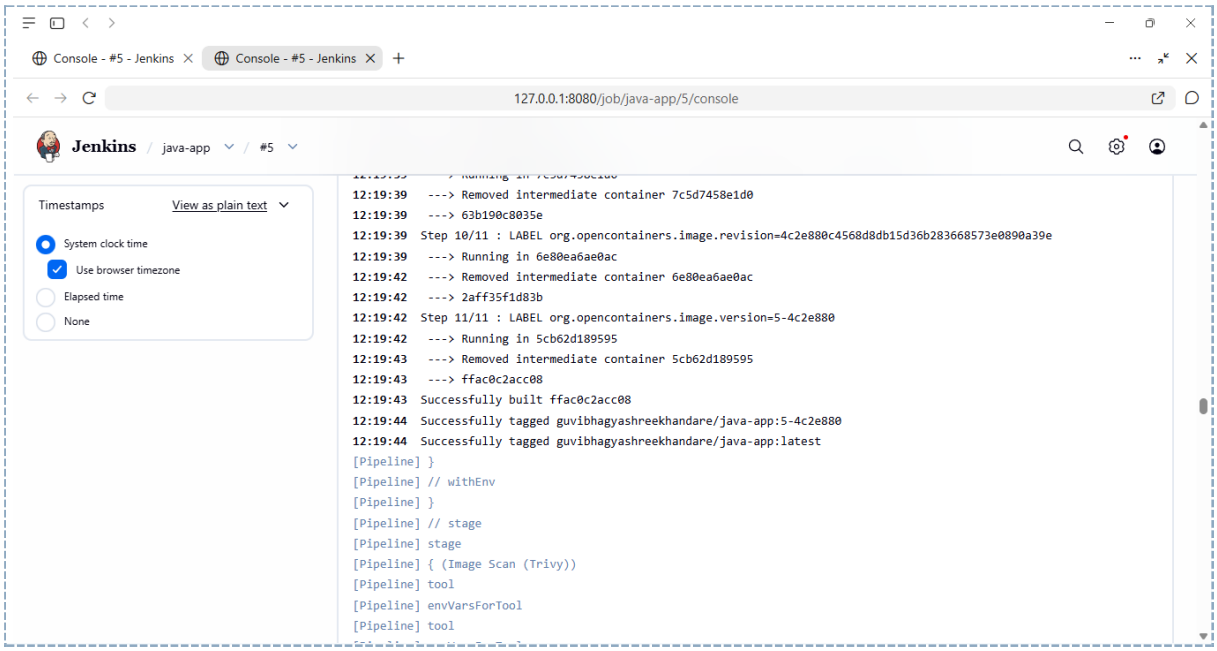


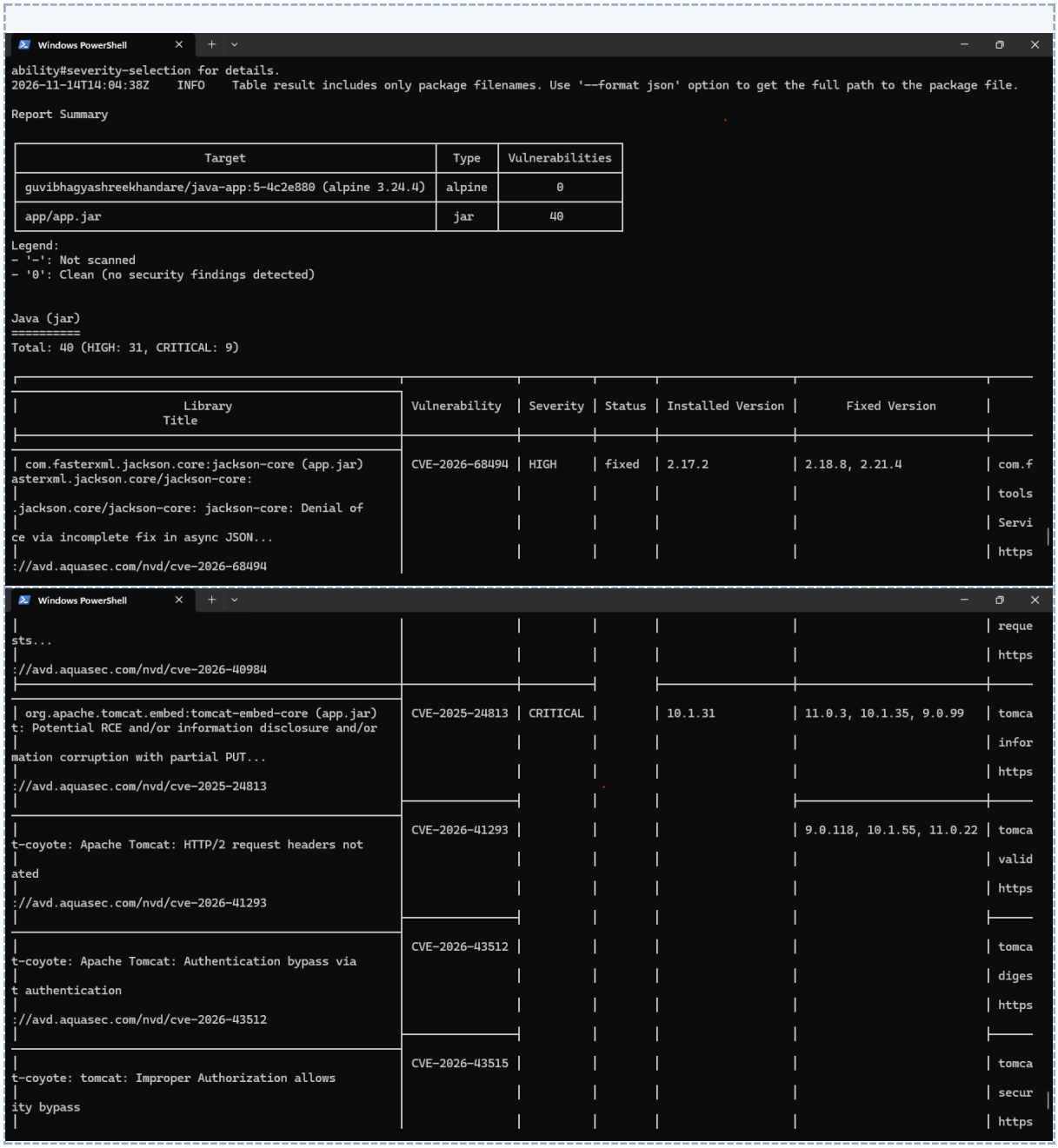
\]'

Screenshot 12: Docker build and Trivy scan output

Jenkins built the image with a unique tag (<build>-<commit>) and Trivy scanned it for HIGH and CRITICAL vulnerabilities.

How to capture: *Build → Console Output, the "Docker Build" and "Image Scan (Trivy)" sections.*



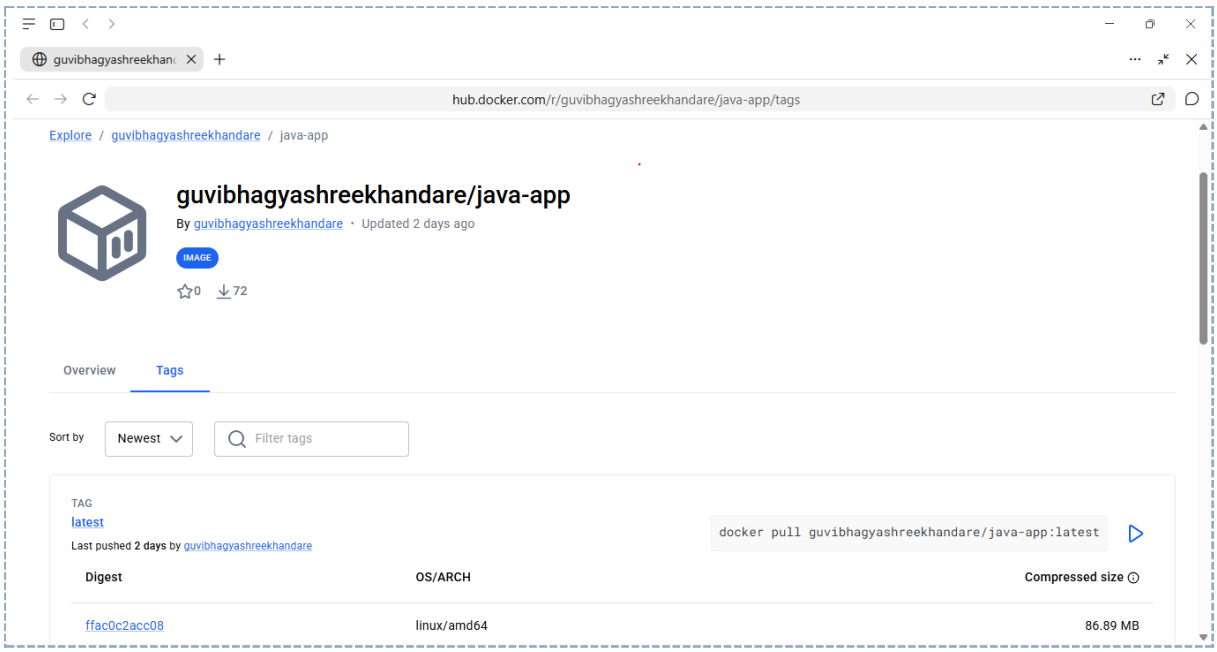

Windows PowerShell
ability#severity-selection for details.
2026-11-14T14:04:38Z INFO Table result includes only package filenames. Use '--format json' option to get the full path to the package file.
Report Summary
Target | Type | Vulnerabilities
guvibhagyashreekhandare/java-app:5-4c2e880 (alpine 3.24.4) | alpine | 0
app/app.jar | jar | 40
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
Java (jar)
==========
Total: 40 (HIGH: 31, CRITICAL: 9)
Library | Vulnerability | Severity | Status | Installed Version | Fixed Version | Title
com.fasterxml.jackson.core:jackson-core (app.jar) | CVE-2026-68494 | HIGH | fixed | 2.17.2 | 2.18.8, 2.21.4 | com.fasterxml.jackson.core/jackson-core: jackson-core: Denial of service via incomplete fix in async JSON... | https://avd.aquasec.com/nvd/cve-2026-68494
org.apache.tomcat.embed:tomcat-embed-core (app.jar) | CVE-2025-24813 | CRITICAL | 10.1.31 | 11.0.3, 10.1.35, 9.0.99 | tomcat: Potential RCE and/or information disclosure and/or information corruption with partial PUT... | https://avd.aquasec.com/nvd/cve-2025-24813
CVE-2026-41293 | 9.0.118, 10.1.55, 11.0.22 | tomcat-coyote: Apache Tomcat: HTTP/2 request headers not validated | https://avd.aquasec.com/nvd/cve-2026-41293
CVE-2026-43512 | tomcat-coyote: Apache Tomcat: Authentication bypass via digest authentication | https://avd.aquasec.com/nvd/cve-2026-43512
CVE-2026-43515 | tomcat-coyote: tomcat: Improper Authorization allows security bypass | https

5. Container Registry and GitOps Update

Screenshot 13: Image pushed to Docker Hub

The image is stored on Docker Hub with a versioned tag (for example 5-4c2e880) and the latest tag.

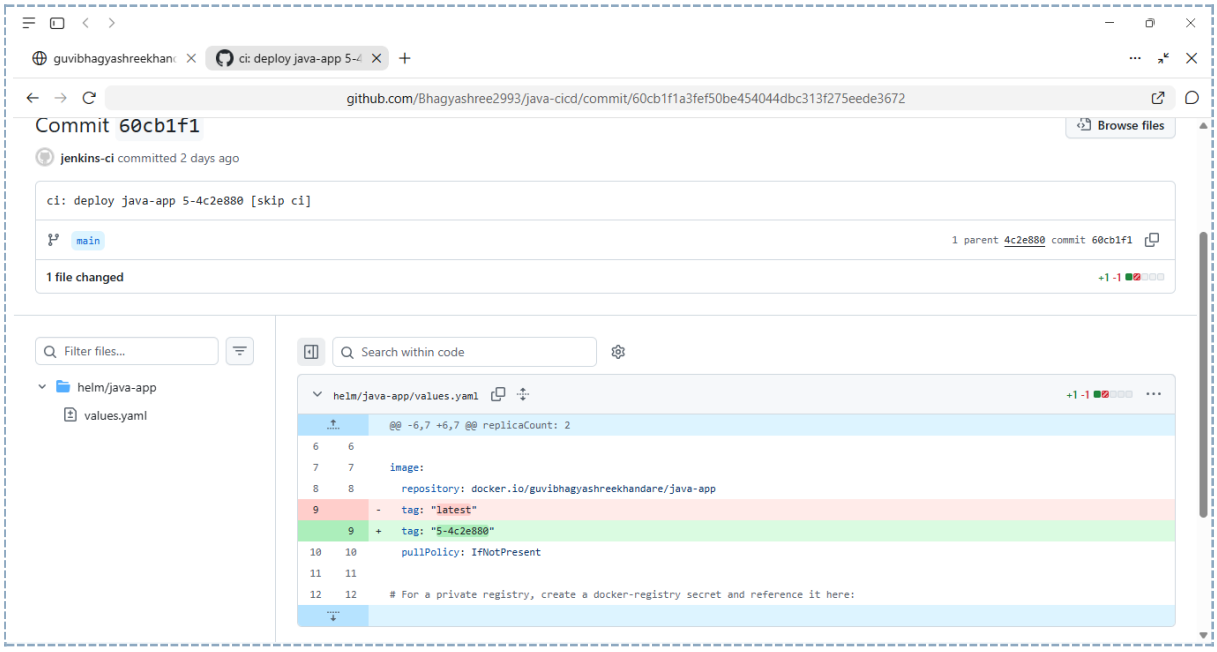
How to capture: *Open https://hub.docker.com/r/guvibhagyashreekhandare/java-app → Tags.*



Screenshot 14: Automatic Helm values commit by Jenkins

Jenkins updated image.tag in helm/java-app/values.yaml and pushed the commit "ci: deploy java-app <tag> [skip ci]". This Git change is what triggers the deployment.

How to capture: *GitHub → Commits → open the "ci: deploy java-app ..." commit to show the changed tag.*



6. Deployment with Argo CD on Kubernetes

Screenshot 15: Kubernetes cluster ready

Docker Desktop Kubernetes is running with one Ready node.

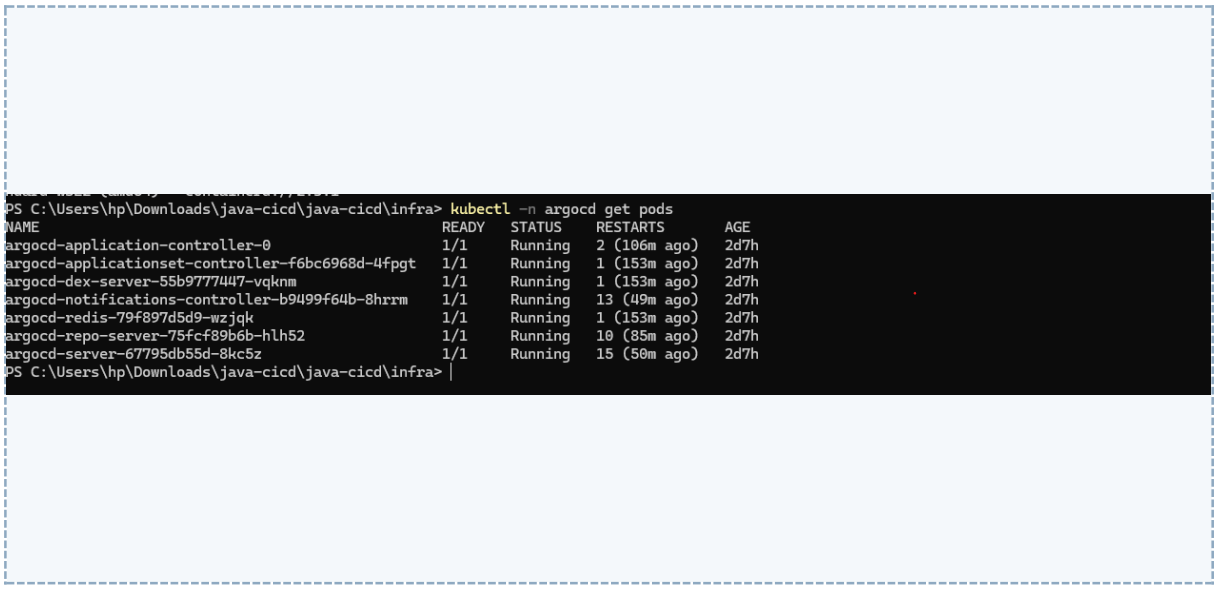
How to capture: *PowerShell:* `kubectl get nodes`



Screenshot 16: Argo CD installed

All Argo CD components are running in the argocd namespace.

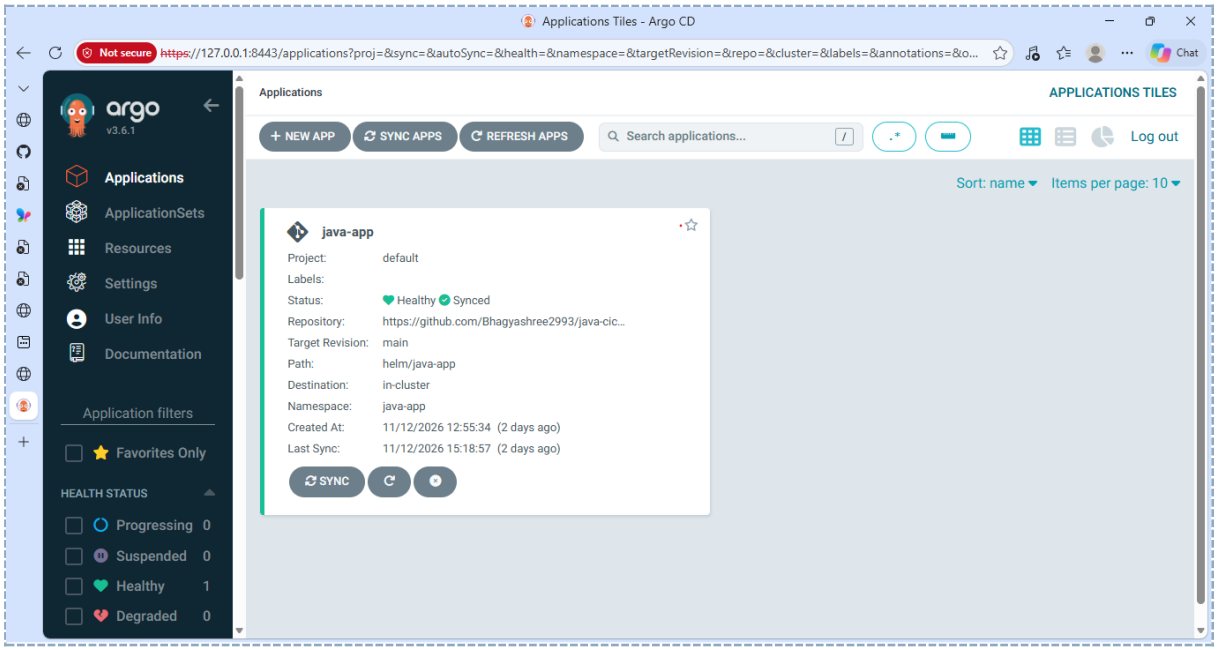
How to capture: *PowerShell:* `kubectl -n argocd get pods`



Screenshot 17: Argo CD application: Synced and Healthy

The java-app Application watches helm/java-app on the main branch and is Synced and Healthy, with automatic sync, prune and self-heal enabled.

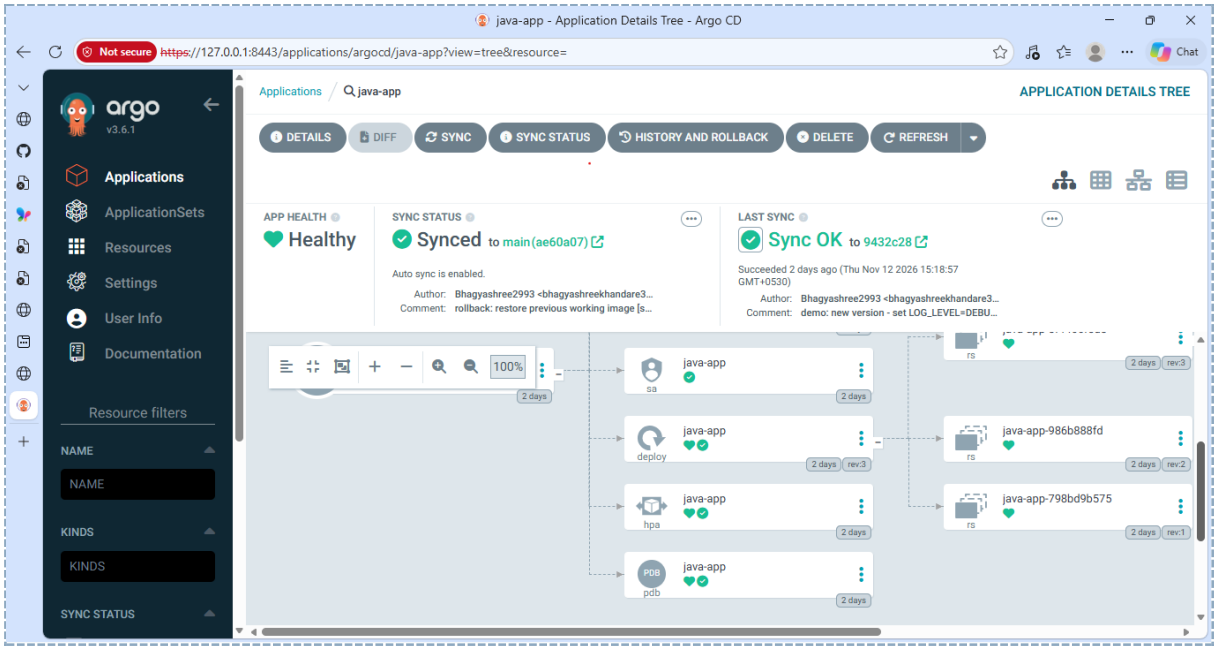
How to capture: *Argo CD UI (https://127.0.0.1:8443) → Applications, or kubectl -n argocd get applications.*



Screenshot 18: Argo CD resource tree

Argo CD shows every Kubernetes object it created from the Helm chart: Deployment, ReplicaSet, Pods, Service, HPA, PodDisruptionBudget and ServiceAccount.

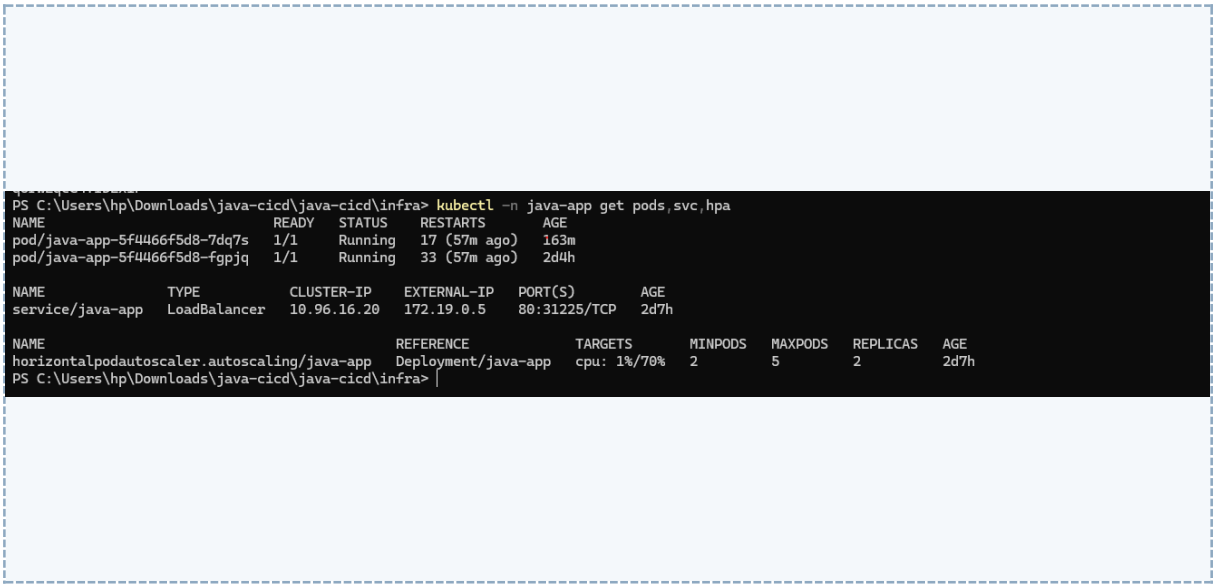
How to capture: *Argo CD UI → click java-app.*



Screenshot 19: Application running in Kubernetes

Two replicas of the application are running behind a Service, with autoscaling configured.

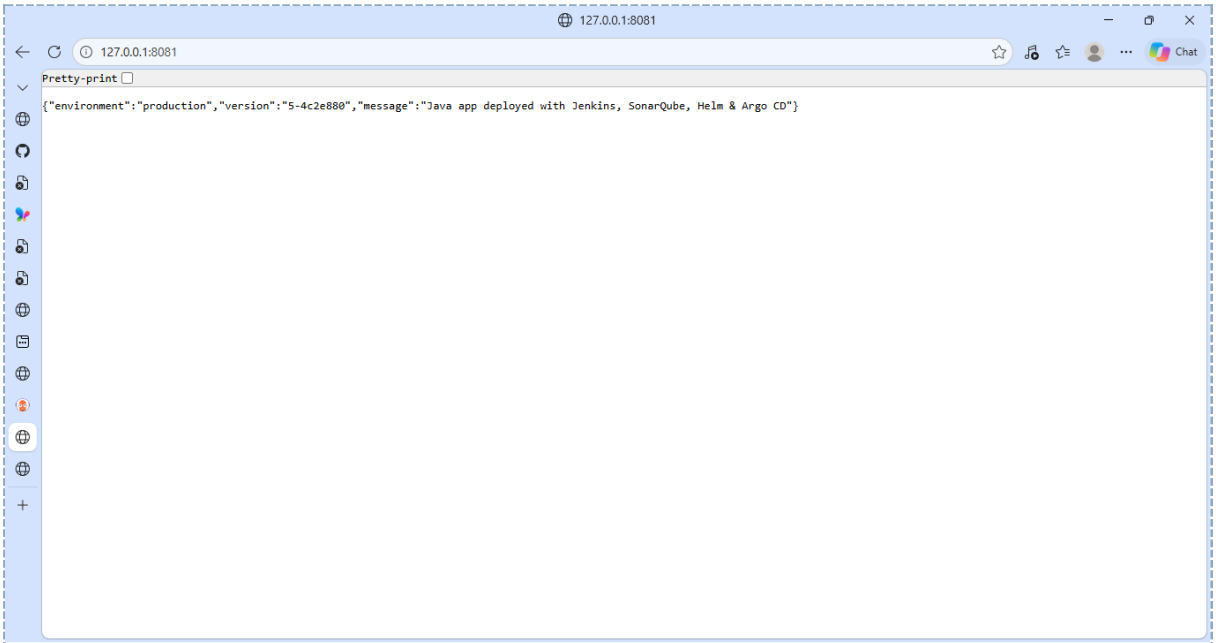
How to capture: *PowerShell:* `kubectl -n java-app get pods,svc,hpa`

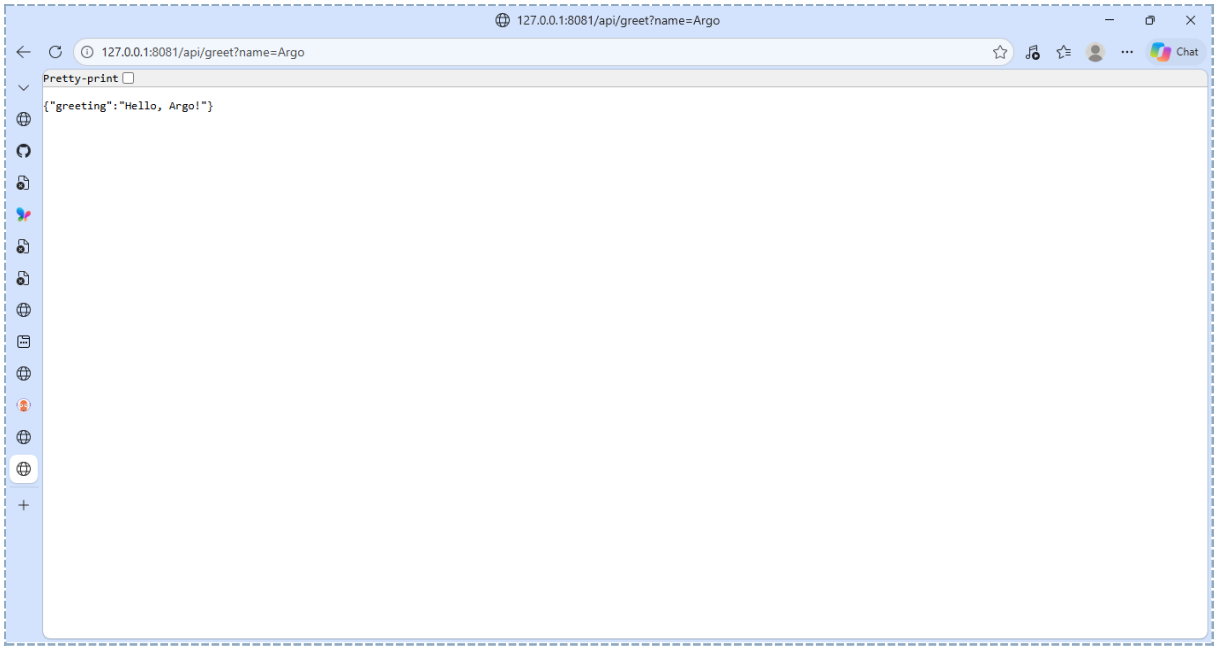


Screenshot 20: Application response in the browser

The deployed application responds and shows the version (image tag) that Jenkins built and Argo CD deployed.

How to capture: *Run* `kubectl -n java-app port-forward svc/java-app 8081:80`*, then open http://127.0.0.1:8081/ and http://127.0.0.1:8081/api/greet?name=Argo*




127.0.0.1:8081/api/greet?name=Argo
127.0.0.1:8081/api/greet?name=Argo
Chat
Pretty-print
{"greeting":"Hello, Argo!"}

7. Challenges Faced and Solutions

Problem	Solution
Docker Desktop not running ("dockerDesktopLinuxEngine" pipe not found)	Started Docker Desktop and waited for "Engine running" before using docker commands.
Docker Hub pulls and pushes failed: "server gave HTTP response to HTTPS client"	Traced it to the network path through Docker Desktop's internal proxy; fixed the proxy and network settings, after which push from Jenkins worked.
Wrong Jenkins opened on localhost:8080 (an older WSL Jenkins)	Used http://127.0.0.1:8080, which reaches the Jenkins container.
Jenkins login lost	Re-enabled sign-up temporarily, created a new admin user, then disabled sign-up again.
SonarQube token saved as the wrong credential type	Stored it as "Secret text" with ID sonar-token, as the SonarQube plugin requires.
Pipeline failed: tool "jdk17" not configured after a restart	Configured JDK 17 (/opt/java/openjdk) and saved it in the main Jenkins configuration.
Jenkins froze during builds (8 GB laptop)	Freed memory by stopping SonarQube while working on Kubernetes and limiting WSL memory.
Trivy scan was slow (database downloaded every build)	Cached the Trivy database in a Docker volume and scanned for vulnerabilities only.
Risk of an endless build loop from the GitOps commit	Jenkins commits with [skip ci] and the Checkout stage skips those builds.

8. Conclusion

The project delivers a complete, automated path from a code change to a running application. Every commit to GitHub is built, tested and checked for quality and security before an image is published. Deployment is fully declarative: the desired version lives in Git, and Argo CD keeps Kubernetes in sync with it, so rollback is as simple as reverting a commit.

Key production practices used

- Quality Gate that blocks low-quality code from being released
- Immutable, uniquely tagged images (build number + Git commit)
- Image vulnerability scanning with Trivy
- Secrets kept in Jenkins credentials, never in code
- GitOps with Argo CD: auto-sync, prune and self-heal
- Non-root, read-only container with liveness, readiness and startup probes
- Zero-downtime rolling updates, autoscaling (HPA) and a PodDisruptionBudget

9. Submission Links

GitHub repository: https://github.com/Bhagyashree2993/java-cicd

Docker Hub image: https://hub.docker.com/r/guvibhagyashreekhandare/java-app